Mid-Decode Model Migration Under a Per-App Memory Limit: Moving a Generation to a Smaller Model Without Holding Both

NagaYu*

Abstract

Prior work first. Closed-form ridge maps between the KV caches of different-size models in one family were introduced by Heo et al. [8]. They apply them at the *prefill hand-off point*. DroidSpeak [9] shares KV caches between models that have the same architecture and are fine-tuned from the same base. This paper adds no new mapping and claims nothing about mapping quality. Its scope is limited to three conditions that this prior work does not address: the switch happens *mid-decode*, between two tokens of one generation; it happens *under memory exhaustion*, triggered by the app's distance to its own per-process limit; and the two models are *never resident together*.

We implement this setting in MLX on Apple silicon and test it against an external jetsam emulator. We compare it with restart, always-small, re-prefill, and the closed-form ridge map of [8] applied mid-decode. That last baseline is reproduced to the letter and runs in the hand-off shape, with both models loaded. On two Qwen2.5 pairs (1.5B→0.5B and 3B→1.5B, 4-bit), a switch that loads the receiver before releasing the sender is killed whenever the app has less spare memory than it needs. Every release-first switch survives. But the transferred cache loses the prompt's specifics. On a needle task with a 3B→1.5B switch just before the answer, the mapped cache answers 0.375 with our additions and 0.583 without, against 1.000 when the small model simply re-reads the transcript. At ∼2 100 tokens the transfer resumes -3.114 s sooner but answers 0.083 against 1.000. In the setting we claim, KV transfer is not the right way to switch, and we state where our own hypothesis fails. iPhone measurements, energy on device, and human ratings are **PENDING:** not yet collected.

1 Scope and prior work

What is not ours. Mapping one model's KV cache into another model's geometry is prior work. Heo et al. [8] fit, for every target layer and KV head, separate key and value maps in closed form by ridge regression ($\lambda = 0.01$). The input is the concatenated KV heads of the top-k source layers, ranked by single-source R^2. Keys have RoPE [12] removed before the map and re-applied after it. The transfer happens at the *prefill hand-off point*: the receiver gets the sender's prefix cache and decodes from there. Their main results are small-to-large over matched KV geometry. Large-to-small is reported only on HellaSwag, where one pair keeps 37% of standalone accuracy. DroidSpeak [9] reuses KV caches between models that share an architecture and a foundation model. It recomputes selected layers and reuses the rest, at prefill time. Re-reading a transcript with a new model (re-prefill) is a standard alternative, and we treat it as a baseline. Keeping the first tokens because attention concentrates on them follows StreamingLLM [13].

*Code, mappers and raw measurements: `https://github.com/NagaYu/molt`. Draft generated October 18, 2026; device results pending (see Section 7).

What this paper claims, and only this. We study one situation. An on-device app is in the middle of generating an answer with a model that nearly fills the app's memory budget. Then something else in the app needs memory. On iOS the app is killed once it crosses its per-process limit [2]. The goal is to continue *the same sentence* with a smaller model, with no retracted tokens and no termination. The contribution is restricted to three conditions:

1. **Mid-decode.** The switch happens between two tokens of one generation. The receiver takes the sender's cache for every token fed so far and itself feeds the token already shown to the user.
2. **Under memory exhaustion.** The switch is triggered by the app's headroom to its own limit (`os_proc_available_memory` [4]), polled before every token, before the limit is crossed.
3. **Non-co-resident.** Sender and receiver weights are never resident together. We show what this requires on MLX [7] and test that the kernel-tracked peak footprint stays under the cap.

Nothing else in this paper is claimed as new. That includes the mapping, its fitting procedure, RoPE handling, re-prefill and sink tokens.

The question that decides whether there is a claim at all. If the simple method is enough in this setting, molt has nothing to claim. The simple method is the closed-form map of [8] applied at the token boundary. We therefore reproduce it without shortcuts (condition C) and measure where it is and is not enough. Section 6 states plainly where our own hypothesis fails.

2 The setting

The limit an app actually faces. Apple documents `os_proc_available_memory()` as "the current memory limit minus the memory footprint of your app". It returns 0 once the limit is exceeded. The limit can change during the app's life cycle, and the value is advisory and should not be cached [4]. A jetsam event report records the reason for a termination. `per-process-limit` means the app crossed the limit imposed on all apps; `vm-pageshortage` means system-wide pressure [2]. The UIKit memory warning [3] and the dispatch memory-pressure source [5] report *system* memory pressure. We found no documentation that promises either one fires before a `per-process-limit` termination. Some devices honour a larger limit under the `increased-memory-limit` entitlement [1]. The quantity the limit is checked against is the physical footprint. On Apple silicon, MLX tensors sit in Metal buffers that the resident set size barely counts. We measured a 1 GiB MLX array moving RSS by ≈0 MiB and the footprint by ≈1 GiB. Every memory number in this paper is therefore phys_footprint, never RSS.

The event. A generation runs on model S, whose footprint leaves headroom h under the limit. At token n_p, the app's other work starts to need B more memory, arriving in chunks of 64 MiB per token. B exceeds h, so S cannot stay, while the smaller model T plus B fits. A condition survives if its footprint never exceeds the limit. A co-resident switch needs about T's weights plus the mapper in spare memory at the moment it starts. A non-co-resident switch needs only the KV caches and the mapper. The interesting band is

$$\underbrace{\text{chunk} + \text{margin}}_{\text{ordered needs}} \;\le\; h \;<\; \underbrace{w_T + w_{\text{map}} + \text{margin}}_{\text{co-resident needs}},$$

where one schedule survives and the other cannot. Section 5.4 measures whether the band exists and how wide it is for real tiers.

3 Method

3.1 The mapping (prior work, condition C)

We reproduce the recipe of [8] as written.

- *Features.* Per source layer, all KV heads are flattened. Keys have the source RoPE removed; values are used as is.
- *Targets.* The destination's own keys (RoPE removed) and values, per layer and head.
- *Regression.* Centred ridge with $\lambda = 0.01$ and bias $b = \bar{y} - \bar{x}W$. Covariance is accumulated in float32 and solved in float64.
- *Data.* 500 FineWeb-Edu [10] sequences of 1 024 tokens, sub-sampled with stride 4.
- *Source selection.* For each destination layer, the top-k source layers are kept by a fixed ranking: head-averaged single-source R^2, averaged over K and V.

Keys are re-rotated with the destination RoPE at the same absolute positions.

We had to make two interpretations, and we state both. (i) The text writes $X^\top X + \lambda I$ without saying whether $X^\top X$ is normalised. On ∼128 K rows an unnormalised Gram matrix would make $\lambda = 0.01$ a no-op, and their reported collapse at $\lambda = 1$ could not happen. We therefore read $X^\top X$ as the centred covariance and also sweep λ (Table 2). (ii) Heo et al. choose k by downstream accuracy. We choose k by held-out prefix-KV negative log-likelihood (NLL), measured on documents disjoint from both the calibration and the evaluation splits.

Implementation checks in `fit.selftest` and `migrate.selftest`: a planted linear map is recovered to $R^2 = 1$; the R^2 computed from sufficient statistics equals the R^2 of actually applying the fitted mapper to within 10^{-7}; and with an identity mapper, transplanting reproduces a native re-prefill to within 1.4×10^{-6} on the test models. On the real 4-bit Qwen2.5-0.5B, mapping its own cache onto itself through the full RoPE strip-and-reapply path changes the logits by at most 0.110 (fp16 rounding). At experiment Q's switch point it gives the same 8-token greedy continuation as native decoding on 24 of 24 needle items.

3.2 Mid-decode transplant

The decode loop keeps one invariant: the cache holds KV for exactly the tokens fed so far, and `pending` is the token already shown to the user but not yet fed. At a switch the receiver gets the mapped cache and feeds `pending` itself, so no shown token is retracted. Molt adds two optional *anchors*, which we treat as engineering choices and not as contributions.

- The first s positions are replaced by the receiver's native KV. This is exact, because causal positions depend only on earlier tokens; it follows the sink observation of [13].
- The last t positions are re-fed natively over the mapped history.

Setting $t =$ all gives re-prefill (condition A′). s and $t \leq 32$ are chosen on held-out text only.

3.3 The memory order (the claim)

Releasing a model on MLX is not one call. We measured three facts (Appendix A). (1) After `del`, buffers stay in MLX's allocator cache and are not reused for arrays of other sizes. (2) `mx.clear_cache()` right after `del` can free nothing, because a buffer returns to the cache only after the GPU has finished the last command buffer that used it. (3) After synchronising and clearing, the OS still reclaims the memory asynchronously, in about 10^2 ms. The *ordered* schedule therefore runs as follows:

1. snapshot the source KV;
2. drop the source weights, synchronise, clear, and poll phys_footprint until it has fallen by 90% of the source weight bytes;
3. load the mapper and map;
4. drop the mapper and source KV the same way;
5. load the receiver;
6. apply the anchors and resume.

Whether steps (2) and (4) are needed at all turns out to depend on MLX's cache limit (Section 5.4). Table 5 lists the three alternatives we measure against it. *colocated* is the prefill hand-off shape. *del-only* is the natural "drop it, then load". *clear-no-wait* is the call a developer would write.

The trigger. Before every token the loop reads the headroom. On iOS that is the return value of `os_proc_available_memory()`; on the Mac it is cap − phys_footprint. It migrates when the headroom falls below what *the chosen schedule* needs plus the next demand chunk. Each schedule thus gets its own best trigger, and a co-resident schedule is never killed merely for reacting late.

4 Experimental setup

Models. We use Qwen2.5 Instruct [11] in the 4-bit MLX builds an app would ship. The pairs, large to small (the direction memory pressure forces), are 3B→1.5B (matched KV geometry: 2 KV heads, $d_h = 128$) and 1.5B→0.5B (mismatched: d_h 128→64). Mappers are fitted on these exact 4-bit weights. All three sizes share one tokenizer, which we checked by vocabulary hash. Decoding is greedy, so conditions differ only in what the switch did.

Conditions. All conditions run on one machine. Each run is a separate process, so no condition inherits another's allocator state.

- **A** restart: discard the generation and start over on the small model.
- **A′** re-prefill: release, load small, and re-read the prompt plus everything shown so far. This baseline is added because it is the obvious alternative; it is not one of the four named in the brief.
- **B** always small.
- **C** the closed-form map of [8] applied at the token boundary, with the selected k^*, in hand-off shape (colocated) and without anchors.

- **D** molt: the same mapper, the ordered schedule and the held-out-selected anchors.

Ablations change one factor each: C+ordered isolates the schedule, D+colocated the anchors, D with the depth-linear mapper the mapper. D with del-only and D with clear-no-wait test the naive freeing variants.

Jetsam emulation. A separate C process polls the worker's `proc_pid_rusage` at about 1 kHz. It records the instantaneous phys_footprint and the kernel ledger's lifetime maximum (`ri_lifetime_max_phys_footprint`), and sends `SIGKILL` the moment either exceeds the cap. The lifetime maximum cannot miss a spike between samples. Phase labels written by the worker attribute every sample to a phase.

Experiments. *I (intrinsic)*: teacher-forced NLL of 128 held-out tokens after the switch, on the FineWeb-Edu evaluation split, at boundary positions $P \in \{32, 128, 256, 512, 768\}$. *Q (quality)*: no cap, with the switch at a fixed token per item. The suite has GSM8K [6] (switch at token 32, mid-reasoning), a needle task with 14 similar records (switch right before the answer value), and open-ended explanations (switch at token 48). *M (memory)*: a cap of $S+h$, where S is the measured footprint of the big model decoding, swept over h; demand arrives as in Section 2.

Metrics.

- **Survival**: the run was not killed.
- **Lost tokens**: tokens shown and then retracted.
- **Visible seam**: the jump in judge NLL across the boundary, $\overline{\mathrm{NLL}}[b, b{+}16) - \overline{\mathrm{NLL}}[b{-}16, b)$, reported relative to A′; and human ratings, which are pending.
- **Continuation quality**: task accuracy, with a killed run scoring 0 and never dropped, plus judge top-1 agreement after the boundary. The judge is the pair's large model.
- **Time to resume**: from trigger to the next shown token. For A, the time until new content appears.
- **Peak footprint during migration**: kernel lifetime maximum.
- **Energy**: CPU energy per process (`ri_energy_nj`) and GPU energy system-wide from IOReport minus an idle baseline (Appendix B).

Throughput is recorded but is not a primary metric. Every aggregate carries a 95% bootstrap interval, paired by item where two conditions are compared.

Platform. Reported per run in the result files: Apple M2, 16 GB, macOS 27.0.1, MLX 0.31.2, mlx-lm 0.31.3. Other users' processes were running during the measurements and swap was nearly full (Section 7). Timings are reported but should be read with that in mind; footprint and survival are not affected.

5 Results

5.1 The prior mapping, reproduced

Figure 1 and Table 1 cover every fitted mapper.

Table 1: Every fitted mapper. Held-out = documents disjoint from calibration and evaluation.

pair	mapper	k	params (M)	MiB	R^2 in	R^2_K held	R^2_V held	NLL held
3B→1.5B	depth_linear	1	3.7	7	0.524	0.640	0.391	3.238
3B→1.5B	heo_k1	1	3.7	7	0.578	0.683	0.456	2.879
3B→1.5B	heo_k2	2	7.4	14	0.645	0.735	0.537	2.713
3B→1.5B	heo_k4	4	14.7	28	0.696	0.770	0.597	2.629
3B→1.5B	heo_k8	8	29.4	56	0.735	0.795	0.642	2.585
3B→1.5B	heo_k16	16	58.7	112	0.763	0.810	0.667	2.556
3B→1.5B	heo_kall	36	132.1	252	0.794	0.819	0.685	2.539
1.5B→0.5B	depth_linear	1	1.6	3	0.592	0.702	0.468	4.285
1.5B→0.5B	heo_k1	1	1.6	3	0.616	0.720	0.497	3.326
1.5B→0.5B	heo_k2	2	3.2	6	0.682	0.769	0.578	3.096
1.5B→0.5B	heo_k4	4	6.3	12	0.723	0.797	0.629	2.996
1.5B→0.5B	heo_k8	8	12.6	24	0.760	0.820	0.671	2.948
1.5B→0.5B	heo_k16	16	25.2	48	0.787	0.834	0.697	2.923
1.5B→0.5B	heo_kall	28	44.0	84	0.806	0.842	0.708	2.937

Table 2: Ridge λ sweep at the selected k^* (centred-covariance reading).

pair	λ	held-out NLL [95% CI]
3B→1.5B	0.0	2.529 [2.385, 2.670]
3B→1.5B	0.001	2.522 [2.379, 2.663]
3B→1.5B	0.01	2.514 [2.372, 2.652]
3B→1.5B	0.1	2.531 [2.392, 2.666]
3B→1.5B	1.0	2.634 [2.495, 2.768]
1.5B→0.5B	0.0	2.864 [2.719, 3.010]
1.5B→0.5B	0.001	2.865 [2.720, 3.010]
1.5B→0.5B	0.01	2.869 [2.724, 3.014]
1.5B→0.5B	0.1	2.897 [2.750, 3.041]
1.5B→0.5B	1.0	3.144 [2.988, 3.296]

1.5B→0.5B, mismatched KV geometry. Heo et al. list this case as future work: head dimension 128→64, 28→24 layers. Calibration used 128,000 rows. On 64 held-out documents, prefix-KV NLL improves monotonically from $k = 1$ (3.326) to $k = 16$ (2.923). At $k =$ all 28 layers it is slightly worse (2.937), so $k^* = 16$ (48 MiB in fp16). For reference, the receiver reading the same prefix itself scores 2.800 and the sender 2.448. The transferred cache therefore costs 0.123 nats per token *more* than letting the small model re-read the text. Held-out R^2 at k^* is 0.834 for keys and 0.697 for values. Values are consistently the harder target.

molt v0.1's own layer pairing (depth-linear, one source layer per target layer) has nearly the same R^2 as Heo's $k = 1$. That is 0.702/0.468 for depth-linear against 0.720/0.497 for $k = 1$. Its NLL, however, is far worse: 4.285 against 3.326. *Which* layers feed a target matters more than the per-layer R^2 suggests, and the prior method's R^2 ranking gets it right where v0.1 did not.

λ and anchors (on a 32-document subset of the held-out split, so absolute values differ from the 64-document sweep). NLL is flat from $\lambda = 0$ (2.864) through $\lambda = 0.01$ (2.869). It degrades at 0.1 (2.897) and collapses at 1 (3.144) (Table 2). This is the shape Heo et al. describe for their own sweep, which supports reading $X^\top X$ as a covariance. Native anchors help, but only a little. With no anchors the NLL is 2.869; the best setting within the 32-token tail budget is $s = 4$,

Table 3: Experiment Q. Accuracy counts every item; needle$_{2K}$ embeds the same task in ∼2 100 tokens of unrelated text. jump = judge-NLL jump at the switch (16 tokens each side); resume = time until new content (all tasks).

pair	condition	GSM8K	needle	needle$_{2K}$	agree$_{post}$	jump	lost	resume (s)
3B→1.5B	big (no pressure)	0.55	1.00	1.00	0.998	–	0	–
3B→1.5B	A restart	0.40	1.00	–	0.861	–	28	1.66
3B→1.5B	A′ re-prefill	0.50	1.00	1.00	0.887	+0.247	0	1.48
3B→1.5B	B always small	0.45	1.00	1.00	0.884	–	0	–
3B→1.5B	C ridge mid-decode	0.15	0.58	0.08	0.672	+2.794	0	0.74
3B→1.5B	D molt	0.45	0.38	0.08	0.769	+1.721	0	1.12
3B→1.5B	C + ordered	0.15	0.58	–	0.698	+2.344	0	1.46
3B→1.5B	D, depth-linear	0.10	0.00	–	0.708	+2.385	0	1.32
1.5B→0.5B	big (no pressure)	0.45	1.00	1.00	0.998	–	0	–
1.5B→0.5B	A restart	0.00	0.96	–	0.860	–	29	1.76
1.5B→0.5B	A′ re-prefill	0.25	0.96	0.42	0.848	+0.724	0	1.38
1.5B→0.5B	B always small	0.00	0.96	0.42	0.850	–	0	–
1.5B→0.5B	C ridge mid-decode	0.00	0.21	0.08	0.674	+1.940	0	1.49
1.5B→0.5B	D molt	0.05	0.12	0.00	0.720	+1.979	0	1.96
1.5B→0.5B	C + ordered	0.00	0.21	–	0.686	+1.779	0	1.43
1.5B→0.5B	D, depth-linear	0.05	0.00	–	0.720	+3.107	0	1.52

$t = 32$ at 2.844. A 128-token tail reaches 2.813, and re-reading everything reaches 2.761. On this intrinsic measure no partial-native setting closes the gap to re-prefill.

3B→1.5B, matched KV geometry (2 KV heads, $d_h = 128$; the setting Heo et al. study). NLL improves with every increase in k, from 2.879 at $k = 1$ to 2.539 at $k =$ all 36 source layers. So $k^* = 36$, and the mapper is 252 MiB in fp16, a sizeable fraction of the receiver itself. The receiver reading natively scores 2.448, so the transfer costs 0.091 nats per token. That is less than for the mismatched pair, but still a cost against re-reading. Depth-linear is again far behind: 3.238. The λ sweep is flat to 0.1 (2.529–2.531) and rises at 1 (2.634). Anchors behave as before: 2.513 with none, 2.483 at $s = 4$, $t = 32$, and 2.424 for a full re-read.

5.2 Transfer cost against switch position

1.5B→0.5B. Teacher-forced, the transferred cache is never as good as letting the receiver read the text itself, and the gap grows with context (Figure 2). For C the gap is +0.023 nats per token at $P = 32$ and +0.146 at $P = 768$, CI [+0.111, +0.196]. molt's anchors narrow it to +0.117 at $P = 768$. At $P = 32$ the 32-token native tail covers the whole context, so D *is* a re-prefill there (+0.000). Per token these gaps look small: about a tenth of a nat, against the 2.448→2.800 difference between the two models. Section 5.3 shows they are not small once the receiver generates freely.

3B→1.5B. The matched-geometry pair shows the same shape with smaller gaps. C costs +0.033 at $P = 32$ and +0.096 at $P = 768$, CI [+0.073, +0.122]. D costs +0.067 at $P = 768$, CI [+0.044, +0.095].

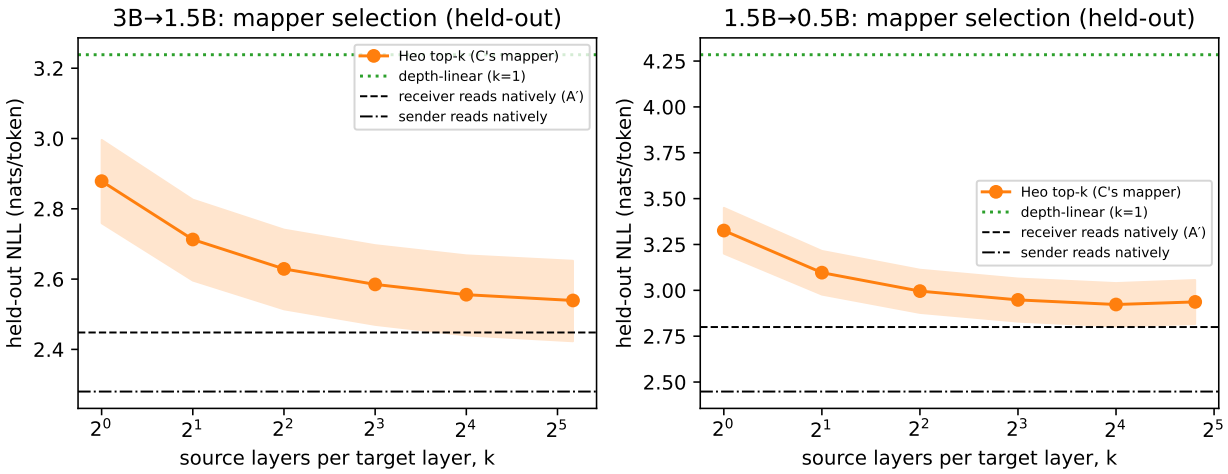


Figure 1: Held-out prefix-KV NLL against the number of source layers k used by the closed-form mapper of [8], with 95% CIs. Dashed: the receiver reading the same prefix natively (what re-prefill achieves). Dash-dot: the sender reading it natively.

5.3 Quality when the switch lands mid-decode

1.5B→0.5B: the transferred cache loses the prompt's specifics. Every migrating condition switches at the same token (Figure 3 and Table 3).

- **Needle** (the switch lands just before the value is written). Re-prefill A′ answers 0.958 of the items correctly, the same as the small model alone (B, 0.958). C answers 0.208 and molt (D) 0.125. The failures are near-misses, such as "8563" for 8564, or plausible wrong codes: the receiver knows it must write a four-digit code, but not which one.
- **GSM8K** (the switch lands mid-reasoning). The big model alone scores 0.450. A′ scores 0.250, *above* B (0.000), because the big model's first 32 tokens set up a short, correct derivation that the small model continues. C scores 0.000 and D 0.050. Their continuations stay fluent and on-topic but bring in quantities the problem never states. Every C and D run hits the 256-token cap without finishing (20/20 and 20/20, against 12/20 for A′).
- **Judge agreement after the switch** (all 80 items). A′ 0.848, C 0.674, D 0.720. The seam jump in judge NLL at the boundary is 0.724 for A′, 1.940 for C and 1.979 for D.

Paired by item:

- D improves on C. Agreement +0.047, CI [0.016, 0.078]; seam 0.040, CI [-0.289, 0.402]. Task score is unchanged (-0.036, CI [-0.107, 0.036]).
- D is far below A′. Task -0.518, CI [-0.661, -0.375]; agreement -0.128, CI [-0.171, -0.089]; seam +1.255, CI [0.834, 1.720].

A restart (A) produces exactly B's text, because decoding is greedy, and before that it retracts every token already shown: 8, 32 or 48 depending on the task, 29.2 on average.

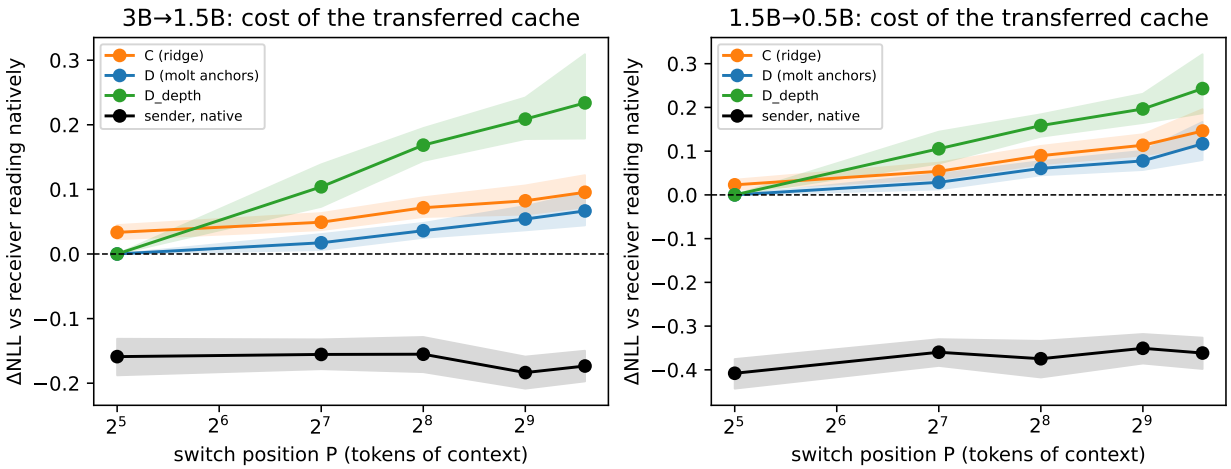


Figure 2: Experiment I. Extra NLL of the receiver decoding from a transferred cache, relative to reading the same text natively, as a function of how much context precedes the switch. Bands are paired 95% CIs over evaluation documents.

3B→1.5B: the same direction, with a stronger receiver.

- **Needle.** A′ and B both score 1.000. C scores 0.583 and D 0.375.
- **GSM8K.** The big model scores 0.550, A′ 0.500 and B 0.450. C scores 0.150 and D 0.450. This is the one task where D recovers to the small model's own level, while C does not.
- **Agreement after the switch.** A′ 0.887, C 0.672, D 0.769.

Paired by item:

- D against C. Agreement +0.096, CI [0.066, 0.128]; seam -1.073, CI [-1.380, -0.772]. Task score is not distinguishable: 0.018, CI [-0.089, 0.143].
- D against A′. Task -0.482, CI [-0.625, -0.339].

GSM8K runs hit the 256-token cap 17/20 times for C and 17/20 for D, against 1/20 for A′.

At long context the transfer is faster and wrong. The $needle_{2K}$ items place the same 14-record list and question around ∼2109 tokens of unrelated FineWeb-Edu text. The switch again lands just before the value is written, and twelve items are run per pair.

- **3B→1.5B.** The receiver alone and A′ both answer 1.000; C answers 0.083 and D 0.083. Paired, D trails A′ by -0.917, CI [-1.000, -0.750], while showing new content -3.114 s sooner, CI [-3.350, -2.890].
- **1.5B→0.5B.** The 0.5B model struggles at this length even reading natively: A′ 0.417, B 0.417. With the transferred cache it is worse still: C 0.083, D 0.000.

The time the transfer saves is real, about 3 s for the 3B pair. What it saves is the time needed to get the answer right.

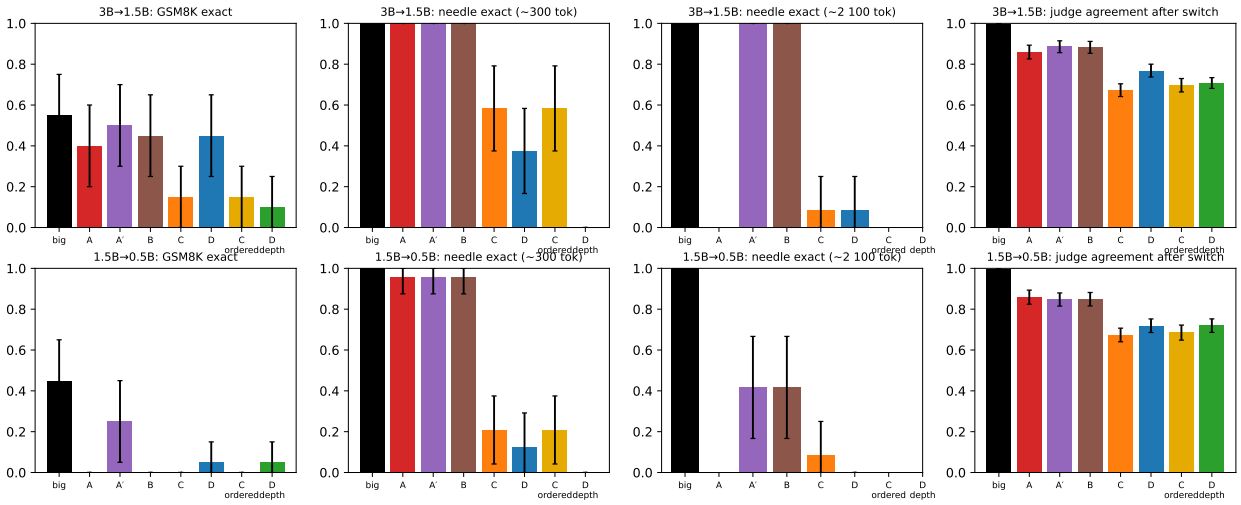


Figure 3: Experiment Q. GSM8K and needle exact match, and judge top-1 agreement after the switch, per condition. Error bars are 95% bootstrap CIs.

5.4 Survival under a per-process limit

The band exists where the arithmetic says, for both pairs. Figure 4 and Table 4 report 3 runs per cell, each in its own process under the watchdog.

1.5B→0.5B. The big model decodes at $S = 1374$ MiB. A co-resident switch needs ≈409 MiB of headroom: 265 MiB of receiver weights, the 48 MiB mapper and a margin.

3B→1.5B. $S = 2235$ MiB, and the need is ≈1176 MiB (828 MiB of weights and a 252 MiB mapper).

Doing nothing (*big, ignores pressure*) is killed in every cell of both pairs, so the emulator reproduces the kill. The hand-off shape (C, and D in the colocated schedule) is killed in `load_dst` or `transplant` in every cell whose headroom is below its need. Examples are 0/3 at $h = 345$ and 0/3 at $h = 1112$. In every cell above the need it survives: 3/3 at $h = 569$ and 3/3 at $h = 1336$. Every release-first schedule survives every cell of both pairs: A, A′, D and C+ordered. At $h = 1112$ for 3B→1.5B, D's kernel lifetime-maximum footprint is 3200 MiB under a 3347 MiB cap. Figure 5 shows the footprint through one switch.

The naive releases fail only under MLX's default allocator. With MLX's default buffer cache, D with del-only is killed in every cell of both pairs, including $h = 2353$ for 3B→1.5B (0/3). D with clear-no-wait is killed in every 3B→1.5B cell and is erratic for 1.5B→0.5B: 3/3, 3/3, 0/3, 3/3 and 3/3 across the grid. This matches Appendix A: `clear_cache` sometimes frees and sometimes does not. With the 20 MiB cache limit that mlx-swift's examples set on iOS, both variants survive every cell of both pairs. For example, del-only is 3/3 at $h = 1112$, while C is still 0/3.

Time to resume: re-prefill wins at the prompt lengths of experiment Q, the transfer wins at ∼2 K tokens. With the short prompts (89 tokens of context at the switch), A′ resumes in 0.66 s and D in 1.17 s for 1.5B→0.5B at $h = 345$. For 3B→1.5B at $h = 1112$ the figures are 0.91 s and 1.48 s. Releasing the sender costs every condition about 0.374 s. D additionally pays 0.509 s in its `map` phase, most of it the second release (mapper and sender KV) and not the mapping itself. A budget-aware planner could skip that release whenever the headroom allows; we did not tune

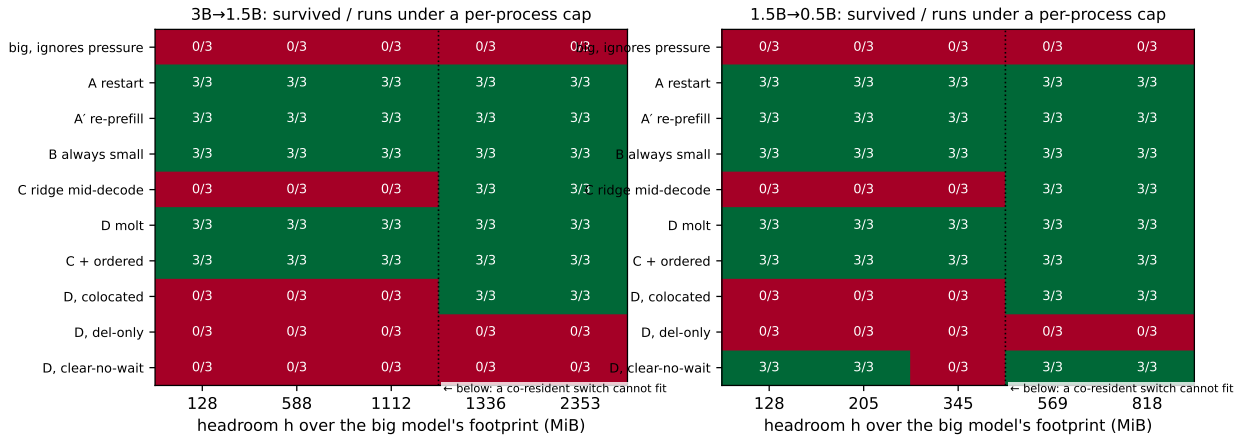


Figure 4: Experiment M. Survival rate against headroom h over the big model's own footprint, one process per run under the jetsam emulator. The dotted line marks the headroom a co-resident switch needs (receiver weights + mapper + margin).

D on these measurements. With ∼2 100-token prompts, the order reverses. For 1.5B→0.5B (cap re-measured on a long prompt, $h = 345$), A′ takes 1.56 s and D 0.97 s. For 3B→1.5B ($h = 1112$), A′ takes 5.17 s and D 1.87 s. C is killed in both (0/3, 0/3). With models resident, mapping compute alone is 1.292 s against 7.738 s for re-prefill at 4 096 tokens (Figure 6).

Energy. CPU energy for the switch (per process) is higher for the transfer in every setting. At short context it is 1.29 J for A′ against 2.01 J for D (3B→1.5B). At long context it is 0.96 J against 1.39 J (1.5B→0.5B). Re-prefill's cost, however, is mostly GPU work, and GPU energy could not be attributed: other processes drew ∼10.5 W of GPU baseline during these runs. We therefore make no energy claim in either direction.

6 Where the hypothesis fails, stated plainly

Where the hypothesis fails, stated plainly. The design hypothesis had two parts. First, carrying the KV cache across the switch would let the small model continue *the same sentence* better than discarding or re-reading the context. Second, molt's additions would make the simple method of [8] sufficient where it alone is not. For both pairs we measured, it does not.

The transferred cache does not carry the content. With a mapped cache (C or D), the receiver keeps the topic and the voice of the text but loses its specifics.

	C	D (molt)	A′ (re-read the text)
needle, 1.5B→0.5B	0.208	0.125	0.958
needle, 3B→1.5B	0.583	0.375	1.000
needle at ∼2 100 tokens, 3B→1.5B	0.083	0.083	1.000

molt's native anchors make the seam smaller and judge agreement higher than C's in both pairs, for example +0.096, CI [0.066, 0.128]. Its task score is not distinguishable from C's (-0.036, CI [-0.107, 0.036]; 0.018, CI [-0.089, 0.143]). "Better than C" and "good enough" are different claims,

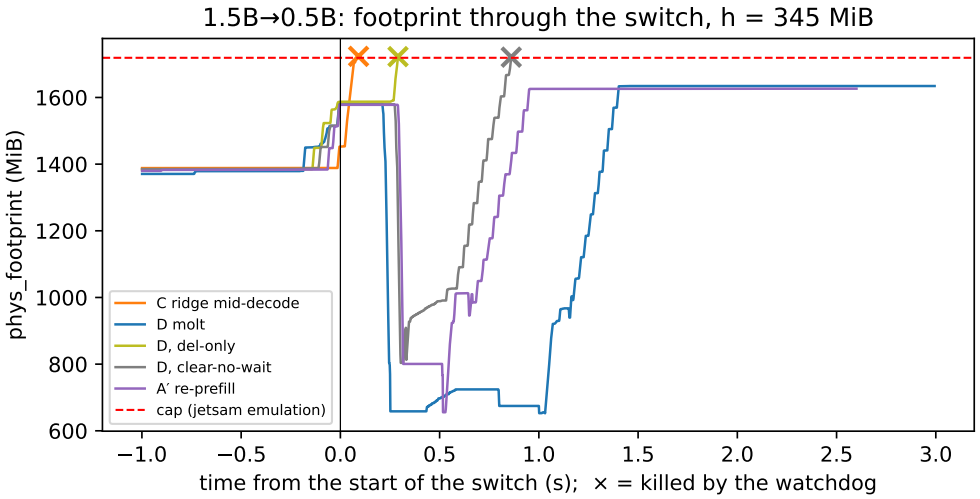


Figure 5: Experiment M, 1.5B→0.5B at $h = 345$ MiB (inside the band). phys_footprint sampled at ∼1 kHz by the external watchdog, aligned at the start of the switch; × marks the watchdog's kill.

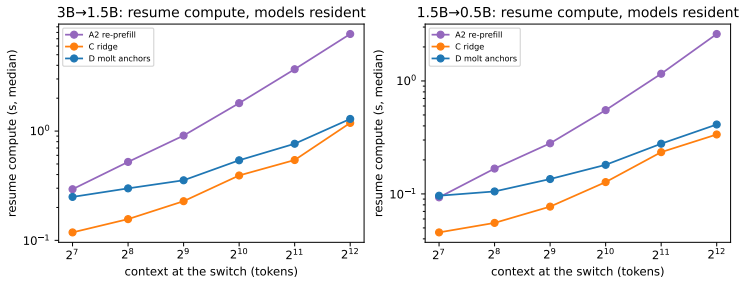


Figure 6: Experiment R. Compute to resume as a function of context at the switch, with both models resident (load costs excluded; median of 5 interleaved repeats).

and only the first one holds here. This is not an implementation fault. The same code path with an identity map on one model reproduces native decoding on 24 of 24 needle items (Section 3.1). The teacher-forced gap of about a tenth of a nat per token (Section 5.2) badly understates what is lost once the receiver generates freely.

Where the transfer wins, it wins at the wrong thing. At the prompt lengths of our main quality runs (89–298 tokens), re-prefill also resumes faster. For 3B→1.5B the times are 0.91 s for A′ and 1.48 s for D, because D pays a second release. At ∼2 100 tokens the transfer is faster: 1.87 s against 5.17 s. On the same long context, however, it answers the needle 0.083 where re-prefill answers 1.000. The seconds it saves are the seconds needed to get the answer right.

The careful release is a fix for one allocator setting, not a general requirement. Under MLX's default buffer cache, drop-then-load is killed in every cell of both pairs, and a plain `clear_cache()` survives erratically or not at all. There, synchronise-clear-wait is what makes the ordered schedule survive. Under the 20 MiB cache limit that mlx-swift's examples set on iOS, both naive variants survive every cell we ran. What holds under both settings is only the *order*: do not

load the receiver before the sender is released. That is a necessary condition, not a technique.

Nothing of molt v0.1's mapper survives. Its depth-linear layer pairing scores held-out NLL 4.285 and 3.238, against 2.923 and 2.539 for the prior method's R^2-ranked top-k. It scores 0 on the needle in both pairs.

What does hold. When the app's headroom is below what a co-resident switch needs (409 and 1176 MiB here), the prefill-hand-off shape is killed. A release-first switch completes, mid-decode, with no retracted token, under both allocator settings and at both context lengths. For what to put in the receiver's cache at that moment, every measurement we have favours the transcript over the transferred KV.

7 Limitations

- **No iPhone results yet.** Every number in this draft comes from an M2 Mac with a jetsam *emulator*. The device protocol is written and its platform core is self-tested on macOS (`ios/`). It covers the Memory Budget Test that reproduces real kills, Instruments traces, and at least three device generations. The app itself has not been compiled or run, because the build machine's Xcode license is not accepted and no device is attached. The simulator is not used: it enforces no per-app limit. Real limits, the `increased-memory-limit` entitlement, and iOS's own reclaim timing could all move the band of Section 5.4.
- **A shared, overloaded machine.** Other users' processes kept swap nearly full throughout. Footprint, survival and quality are unaffected. Times and GPU energy are inflated and noisy, and are reported with the machine's load average.
- **Energy is partial.** CPU energy is per process. GPU energy is system-wide minus an idle baseline. DRAM energy is missing. A `powermetrics` cross-check is pending.
- **Human ratings are pending.** The seam is measured only with a model judge, which is the pair's own large model.
- **Scope.** One model family (Qwen2.5), 4-bit weights, greedy decoding, English text.

8 Conclusion

We moved a generation from a larger to a smaller model mid-decode, triggered by the app's headroom, without ever holding both models. We compared this against a faithful reproduction of the closed-form KV map of [8], on two Qwen2.5 pairs and at two context lengths.

- **One result holds.** The memory order decides survival. Loading the receiver before releasing the sender, as a prefill hand-off does, is killed whenever the headroom is below the receiver plus the mapper. Releasing first survives.
- **The mapping did not carry the prompt's content.** This holds with and without our anchors.
- **Re-reading the transcript was better on every quality measure.** The transfer resumes faster only at long context, and there it loses the answer.

For an on-device app that reaches its memory limit mid-answer, our data recommend a release-first re-prefill, not KV transfer. iPhone measurements (Section 7) are the next step, because iOS's own limits and reclaim behaviour could move the band.

References

[1] Apple Inc. com.apple.developer.kernel.increased-memory-limit. Apple Developer Documentation, `https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.increased-memory-limit`, . Accessed 2026-10-17.

[2] Apple Inc. Identifying high-memory use with jetsam event reports. Apple Developer Documentation, `https://developer.apple.com/documentation/xcode/identifying-high-memory-use-with-jetsam-event-reports`, . Accessed 2026-10-17.

[3] Apple Inc. didreceivememorywarningnotification. Apple Developer Documentation, `https://developer.apple.com/documentation/uikit/uiapplication/didreceivememorywarningnotification`, . Accessed 2026-10-17.

[4] Apple Inc. os_proc_available_memory. Apple Developer Documentation, `https://developer.apple.com/documentation/os/os_proc_available_memory`, . Accessed 2026-10-17.

[5] Apple Inc. makememorypressuresource(eventmask:queue:). Apple Developer Documentation, `https://developer.apple.com/documentation/dispatch/dispatchsource/makememorypressuresource(eventmask:queue:)`, . Accessed 2026-10-17.

[6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[7] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. `https://github.com/ml-explore/mlx`, 2023.

[8] Taekyung Heo, Rasoul Shafipour, Ritchie Zhao, Maximilian Golub, Mohammad Mahdi Kamani, Ritika Borkar, Makesh Tarun Chandran, Pantea Zardoshti, and Bita Darvish Rouhani. Cross-model KV cache transfer in LLM families: A closed-form linear mapping for prefill reuse. *arXiv preprint arXiv:2608.03893*, 2026.

[9] Yuhan Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, Madan Musuvathi, and Esha Choukse. DroidSpeak: KV cache sharing for cross-LLM communication and multi-LLM serving. *arXiv preprint arXiv:2411.02820*, 2024.

[10] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

[11] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[12] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[13] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

A What freeing a model costs on MLX

All three facts below are measured by `mlx/molt_mlx/footprint.py::selftest` on every run of the test suite.

1. A 256 MiB MLX array adds 262 MiB to phys_footprint, while total RSS is 32 MiB.
2. After `del` the footprint is unchanged (+262 MiB). `mx.clear_cache()` immediately after `del`, followed by a 300 ms sleep, freed the buffer in only 0 of 5 runs; in the others the footprint stayed up by as much as +262 MiB. Whether it frees depends on whether the GPU has finished with the buffer, so it is not something to rely on.
3. `mx.synchronize()`, then `clear_cache()`, then polling returned the memory every time, with a median of 175 ms (range 40–185 ms).

The peak-footprint test (`mlx/tests/test_peak_footprint.py`) runs each schedule in its own process under a cap of (source-resident footprint + 40 MiB). The models are a synthetic 290 MiB source and a 109 MiB destination, so the cap is below source plus destination. Ordered peaks at 688 MiB under a 725 MiB cap; colocated and del-only are killed in `load_dst`. clear-no-wait survived 3 of 3 runs at this size, so it is *not* shown to fail there; the full-size result is in Section 5.4.

B Energy without root

`ri_energy_nj` counts CPU energy only: a busy MLX matmul loop reads 0.02 W in it. IOReport's "GPU Energy" channel reads 11.58 W for the same loop, with no root required. Energy is reported as these two named parts and never summed silently. DRAM energy is unavailable without `powermetrics`; `bench/powermetrics.sh` is the root cross-check.

C Reproducing every number

```
python -m pytest mlx/tests --import-mode=importlib  # selftests + peak test
python -m bench.facts                      # MLX freeing + energy facts
python -m bench.fit_mappers --pair 1.5b-0.5b   # fit, sweep, select
python -m bench.intrinsic   --pair 1.5b-0.5b   # experiment I
python -m bench.run_memory  --pair 1.5b-0.5b   # experiment M
python -m bench.run_quality --pair 1.5b-0.5b   # experiment Q
python -m bench.score       --pair 1.5b-0.5b   # judge scores
python -m bench.analyze                    # bench/results/summary.json
python figures/make_paper_figures.py       # figures + this paper's macros
python -m bench.make_results_md            # RESULTS.md
cd ios && swift run MoltCoreSelfTest       # Swift core
```

`python -m bench.run_all` runs the middle block for both pairs, resumably. Figures and every number in the text are produced from `bench/results`; none is typed by hand.

Table 4: Experiment M: runs that survived / runs, per condition and headroom h (MiB). Every run is its own process under the watchdog. Peaks, kill phases and times for every cell are in RESULTS.md.

3B→1.5B, default MLX cache; co-resident need ≈1176 MiB					
condition	$h = 128$	$h = 588$	$h = 1112$	$h = 1336$	$h = 2353$
big, ignores pressure	0/3	0/3	0/3	0/3	0/3
A restart	3/3	3/3	3/3	3/3	3/3
A′ re-prefill	3/3	3/3	3/3	3/3	3/3
B always small	3/3	3/3	3/3	3/3	3/3
C ridge mid-decode	0/3	0/3	0/3	3/3	3/3
D molt	3/3	3/3	3/3	3/3	3/3
C + ordered	3/3	3/3	3/3	3/3	3/3
D, colocated	0/3	0/3	0/3	3/3	3/3
D, del-only	0/3	0/3	0/3	0/3	0/3
D, clear-no-wait	0/3	0/3	0/3	0/3	0/3
3B→1.5B, 20 MiB cache limit; co-resident need ≈1176 MiB					
condition	$h = 128$	$h = 588$	$h = 1112$	$h = 1336$	$h = 2353$
A restart	3/3	3/3	3/3	3/3	3/3
A′ re-prefill	3/3	3/3	3/3	3/3	3/3
C ridge mid-decode	0/3	0/3	0/3	3/3	3/3
D molt	3/3	3/3	3/3	3/3	3/3
C + ordered	3/3	3/3	3/3	3/3	3/3
D, del-only	3/3	3/3	3/3	3/3	3/3
D, clear-no-wait	3/3	3/3	3/3	3/3	3/3
1.5B→0.5B, default MLX cache; co-resident need ≈409 MiB					
condition	$h = 128$	$h = 205$	$h = 345$	$h = 569$	$h = 818$
big, ignores pressure	0/3	0/3	0/3	0/3	0/3
A restart	3/3	3/3	3/3	3/3	3/3
A′ re-prefill	3/3	3/3	3/3	3/3	3/3
B always small	3/3	3/3	3/3	3/3	3/3
C ridge mid-decode	0/3	0/3	0/3	3/3	3/3
D molt	3/3	3/3	3/3	3/3	3/3
C + ordered	3/3	3/3	3/3	3/3	3/3
D, colocated	0/3	0/3	0/3	3/3	3/3
D, del-only	0/3	0/3	0/3	0/3	0/3
D, clear-no-wait	3/3	3/3	0/3	3/3	3/3
1.5B→0.5B, 20 MiB cache limit; co-resident need ≈409 MiB					
condition	$h = 128$	$h = 205$	$h = 345$	$h = 569$	$h = 818$
A restart	3/3	3/3	3/3	3/3	3/3
A′ re-prefill	3/3	3/3	3/3	3/3	3/3
C ridge mid-decode	0/3	0/3	0/3	3/3	3/3
D molt	3/3	3/3	3/3	3/3	3/3
C + ordered	3/3	3/3	3/3	3/3	3/3
D, del-only	3/3	3/3	3/3	3/3	3/3
D, clear-no-wait	3/3	3/3	3/3	3/3	3/3

Table 5: Memory schedules. All of them compute the same transfer; they differ only in what is resident when.

schedule	order	resident at peak
colocated	load receiver, map, drop sender	sender + receiver + mapper
del-only	drop sender refs, map, load receiver	sender (cached by MLX) + receiver
clear-no-wait	drop, `clear_cache()`, load at once	depends on reclaim timing
ordered	drop, sync, clear, *wait*, map, drop mapper, load	$\max(\text{sender}, \text{KV} + \text{mapper}, \text{receiver})$